

Four points from the design review

Use-case coverage, what "one schema per account" means, API versus our own servers per model, and the full database schema. Supplements the backend design draft; nothing else in that draft changes.

23 September 2026 · Yahya Mateen · supplement to backend design draft 0.1

1 · Use-case coverage

Status of the backend logic behind each of the 50 use cases. "Built" means the logic exists and has been run against the test mailbox; screens are the frontend's. "Partly" means the pieces exist but a step is untested or missing. "Not built" is billing.

40		7	3	50
built		partly	not built (billing)	total
#	USE CASE	BACKEND	NOTE	
UC-01	Register Account	built	email + password, confirm token; Google/Microsoft sign-in to the account not built	
UC-02	Sign In	partly	email + password with sessions; provider sign-in not built	
UC-03	Start a Subscription	not built	billing	
UC-04	Gmail by App Password	built	the default door; tested	
UC-05	Outlook by Microsoft Sign-In	partly	code exists; needs a Microsoft app registration; untested	
UC-06	Yahoo by App Password	partly	server known; flow untested	
UC-07	Zoho by App Password	partly	server known; flow untested	
UC-08	iCloud by App-Specific Password	partly	server known; flow untested	
UC-09	Any Other IMAP Mailbox	built	server from address, then MX; manual entry not built	
UC-10	Establish Mailbox Connection	built	verify, folders, access level, capabilities	
UC-11	Gmail by Google Sign-In	built	the API door	
UC-12	Upgrade to Google Sign-In	built		

UC-13	Disconnect a Mailbox	built	credential and index erased; what is lost listed
UC-14	Scan Mailbox History	built	envelopes in bulk, resumable
UC-15	Review and Clear the Pile	built	gated
UC-16	Clear a Brand's Mail	built	gated
UC-17	Unsubscribe from a Sender	built	one-click only; outcome watched 14 days
UC-18	Reclaim Storage Space	built	
UC-19	Prioritise Newly Arrived Mail	built	the pipeline
UC-20	Recover Important Mail from Spam	built	never automatic
UC-21	Inspect a Suspicious Message Safely	built	
UC-22	Track Unanswered Outgoing Mail	built	Gmail API door only
UC-23	Record Commitments from Structured Mail	built	
UC-24	Warn About an Approaching Deadline	built	
UC-25	Generate a Reply Draft	built	redact, draft in voice, gated send
UC-26	Send a Quick Answer	built	three fixed answers, gated
UC-27	View the Unified Inbox	built	
UC-28	Search Across All Mailboxes	built	sender, subject, date; not message text
UC-29	Correct a Decision	built	
UC-30	Undo an Action	built	both doors
UC-31	Receive the Weekly Digest	built	
UC-32	Export or Erase My Data	built	
UC-33	Identify a Request and Its Deadline	built	
UC-34	Connect a Calendar	partly	Google only; live sign-in not exercised
UC-35	Forward a Batch to Someone	built	drafts only, recipient from the user
UC-36	Resurface an Unread Important Message	built	

UC-37	Set a Follow-Up Reminder	built	
UC-38	Create a Rule in Plain English	built	interpretation confirmed, then no AI
UC-39	Catch Up After Time Away	built	
UC-40	See Where a Conversation Stands	built	
UC-41	See Everything About One Person	built	
UC-42	Tune My Writing Voice	built	
UC-43	Work Through Today	built	
UC-44	Mark a Person as Important	built	
UC-45	Track What You Promised	built	
UC-46	The First Cleanup	built	
UC-47	Where My Data Is Stored	partly	region recorded and shown; move on request not built
UC-48	Change Plan	not built	billing
UC-49	Cancel Subscription	not built	billing
UC-50	Identify the Mail Provider	built	known endings, then MX records

2 • "One schema per account", clarified

Account, not mailbox. An account is one customer (one sign-in). A customer may connect several mailboxes, say a Gmail and a work Outlook. All of them live in that customer's one schema; the `account` column on each message says which mailbox it came from. So "one schema per account" means one private area per customer, never one per mailbox. Ten thousand customers with three mailboxes each is ten thousand schemas, not thirty thousand.

The layout: a small shared layer (accounts, sessions, mailboxes, encrypted credentials) and one private schema per customer holding everything about that customer's mail, whichever mailbox it came from. Separation is structural: a query pointed at one customer's schema cannot name another's.

3 • API or our own servers, per model

Only one model stays hosted: the reading and drafting model (Gemini). The two small models run on our servers. The tables show why, at each number of users.

For each model, the monthly cost of calling it as a hosted API against the monthly cost of running it on servers we rent, at each number of users. Same assumptions as 8.2.

Classifier and name spotter (278 M parameters each)

USERS	AS API (DEDICATED CPU ENDPOINT, 2 MODELS)	ON OUR SERVERS	CHEAPER
10	\$48 (2 × \$0.033/h, billed 24/7)	€0 extra: 3 GB of the €11 server	ours
100	\$48	€0 extra	ours
1,000	\$48 – 96 (a second replica at peak)	€0 extra: 1.5 CPU-hours a day inside the worker servers	ours
10,000	\$150 – 300 (3 – 6 replicas)	€40: two 8 vCPU / 16 GB servers as a model service	ours

Small models cost almost nothing to run and a fixed fee to rent. They are cheaper on our servers at every size, and the name spotter must stay with us anyway, since its job is to find the private parts before anything leaves.

Reading and drafting (today Gemini Flash-Lite; self-host option: an open 7 – 8 B model on a rented GPU)

USERS	TOKENS / MONTH	AS API (FLASH-LITE \$0.10 IN / \$0.40 OUT PER M)	ON OUR SERVERS (L4 GPU \$0.80/H ≈ \$580 / MONTH EACH)	CHEAPER
10	13 M in · 1.4 M out	\$0 – 2 (free tier)	\$580 (1 GPU, mostly idle)	API
100	130 M in · 14 M out	\$20	\$580 (1 GPU)	API
1,000	1.3 B in · 140 M out	\$190	\$580 (1 GPU, ~50 % busy)	API

10,000	13 B in · 1.4 B out	\$1,900	\$3,500 (6 GPUs)	API
break-even	around 20,000 users at these prices, and only if the open model matches Flash-Lite on our prompts, which is unproven. Batch mode halves the API price and pushes break-even further out.			

GPU throughput assumed ~1,000 tokens/s mixed in and out for a 7 – 8 B model on one L4, ≈ 86 M tokens a day. Spot or reserved GPUs are 30 – 60 % cheaper and move the numbers, not the verdict.

Chat understanding (parked)

USERS	AS API (FLASH-LITE, 10 TURNS A DAY)	ON OUR SERVERS (OUR TWO SMALL MODELS, RETRAINED)	CHEAPER
10	\$0 (free tier)	€0 extra	tie
100	\$3	€0 extra	ours, barely
1,000	\$30	€0 extra	ours
10,000	\$300	€0 – 40	ours

Cost is not what decides this one. The small-model route needs a few hundred labelled example sentences first; the API route needs none. Privacy and dependency favour ours; time to build favours the API.

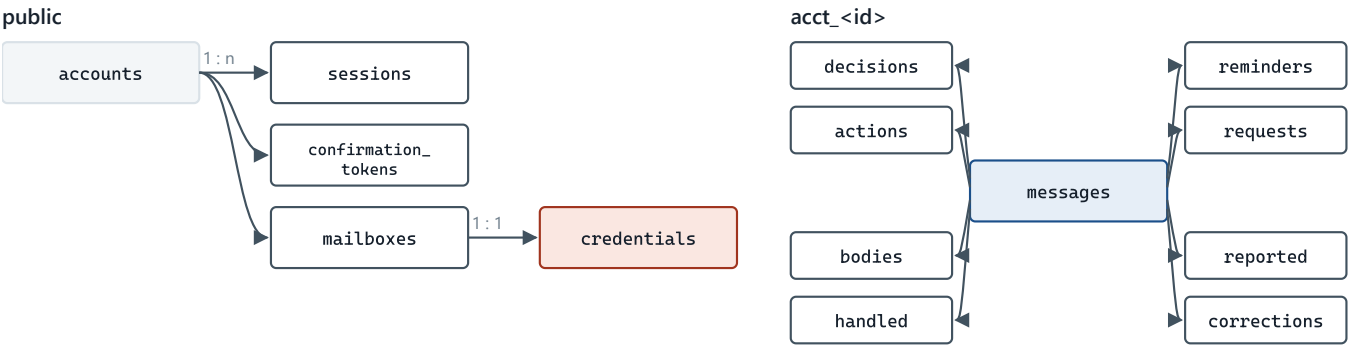
Everything together

USERS	ALL HOSTED APIS	ALL ON OUR SERVERS	MIXED (SMALL MODELS OURS, READING AND DRAFTING BY API) PROPOSED
10	€60	€560	€15
100	€85	€570	€50
1,000	€400	€850	€350
10,000	€2,900	€4,500	€2,900

Servers, database and the model costs above, rounded. "All hosted" rents the small models as endpoints too; "all ours" rents GPUs for reading and drafting. The mixed row is what section 9.2 proposes. Prices September 2026: Gemini 2.5 Flash-Lite \$0.10 / \$0.40 per M tokens; Hugging Face dedicated CPU \$0.033/h, L4 GPU \$0.80/h; Hetzner CPX32 €10.99, CPX42 €19.99; Neon \$0.106/CU-h, \$0.35/GB-month.

4 • Database schema

Every table and column. The shared layer is Postgres DDL in `db/schema.sql`. The per-account tables are the same definitions the proof of concept runs on SQLite, created in Postgres by the store the first time an account is used.



joined by `message_id` (the RFC Message-ID), so the same mail seen through IMAP and the Gmail API is one row

also in the account schema, not keyed by message:
`rules · unsub_requests · person_links · away · saved_recipients · style_profile ·`

4.1 • Shared layer, schema public

accounts • one customer

<code>id</code>	bigserial, key
<code>email</code>	text, unique
<code>password_hash</code>	text, scrypt; null for provider sign-in
<code>confirmed_at</code>	timestampz
<code>data_region</code>	text
<code>plan</code>	text: trial · ...
<code>digest</code>	text: daily · weekly · monthly · never
<code>digest_last</code>	timestampz
<code>created_at</code>	timestampz

sessions • a signed-in browser

<code>token</code>	text, key
<code>account_id</code>	→ <code>accounts.id</code>
<code>created_at</code>	timestampz
<code>expires_at</code>	timestampz

confirmation_tokens • single-use links

token	text, key
account_id	→ accounts.id
purpose	text: confirm · reset
expires_at	timestampz

mailboxes · one connected address

id	bigserial, key
account_id	→ accounts.id
address	text, unique across all accounts
provider	text: gmail · outlook · yahoo · zoho · icloud · other
route	text: app_password · google_signin · microsoft_signin
host	text
access_level	text: full · read only
state	text: connected · paused · disconnected
state_reason	text
folders	jsonb: sent, drafts, spam, trash, archive
capabilities	jsonb
connected_at	timestampz
bookmark	text: history id or highest UID

credentials · the encrypted app password

mailbox_id	→ mailboxes.id, key
kind	text: app_password · oauth_refresh
ciphertext	bytea, encrypted before it arrives
updated_at	timestampz

schema_versions · migrations applied

version	integer, key
applied_at	timestampz
note	text

4.2 · Per-account layer, schema acct_<id>

messages · one row per message, headers only, no body

provider_id	text, key: the provider's id
message_id	text: the RFC Message-ID, the real identity
thread_id	text
account	text: which mailbox it came from
sender	text
sender_name	text
sender_domain	text
recipients	text
subject	text
date	text
date_iso	text, sortable
size	integer, bytes
unread	0/1
in_inbox	0/1
categories	text
bulk	0/1: Precedence bulk
unsubscribe	text: List-Unsubscribe header
one_click	0/1
list_id	text
auto_sub	text: Auto-Submitted
in_reply_to	text
refs	text: References
auth_results	text: SPF/DKIM/DMARC verdict

bodies · message text for a short window, then trimmed

message_id	text, key
provider_id	text
account	text
text	text

chars	integer
fetches_at	text
decisions · what each stage decided and why	
message_id	text
stage	text
decision	text
reason	text
score	real
at	text
key	(message_id, stage)
actions · everything done to the mailbox, with its undo	
id	serial, key
message_id	text
provider_id	text
action	text: label · archive · trash · send · ...
detail	text
before	text: what to restore
undone	0/1
at	text
corrections · the person disagreed	
id	serial, key
scope	text: sender · domain · ...
target	text
was	text
should_be	text
source	text
at	text
handled · replied / drafted, once	

message_id	text
action	text
sent_id	text
at	text
key	(message_id, action)

reminders · follow-ups and deadlines

message_id	text
provider_id	text
kind	text: remind · deadline
due	text
subject	text
why	text
done	0/1
key	(message_id, kind)

requests · asks of the person and promises by them

id	serial, key
message_id	text
provider_id	text
account	text
direction	text: ask · promise
what	text
who	text
due	text
evidence	text
confidence	real
status	text: open · done · dismissed
at	text

rules · typed rules and important-person rules

id	serial, key
sentence	text: as typed
match_kind	text: person · domain · subject
match_value	text
action	text
action_arg	text
plain	text: shown back
group_id	text
at	text

unsub_requests · unsubscribes sent and their outcome

sender	text, key
requested_at	text
outcome	text: watching · stopped · ignoring
checked_at	text

person_links · addresses that are the same person

address	text, key
person	text
evidence	text
by_owner	0/1
at	text

reported · surfaced once, not again

message_id	text
kind	text
at	text
key	(message_id, kind)

away · time-away periods for catch-up

id	serial, key
since	text

until	text
stated	0/1
done	0/1

saved_recipients · addresses the person typed for forwarding

address	text, key
label	text
at	text

style_profile · how the person writes

key	text, key
value	text

style_examples · sent messages used for voice

id	serial, key
text	text
subject	text
embedding	bytea; pgvector in production

Text columns hold ISO dates in the account layer because the same definitions run on SQLite for local development. Sizes measured: about 1 KB per message across all tables.